

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Курсовая работа
по курсу «*Технологии параллельного программирования*»

Обратная трассировка лучей (Ray Tracing) на GPU

Выполнил: Концебалов О.С.

Группа: М80-409Б-22

Преподаватель: Морозов А.Ю.

Москва, 2025

Условие

Цель работы. Использование GPU для создание фотореалистической визуализации. Рендеринг полужеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.

Задание. Вариант: 6.

Сцена. Прямоугольная текстурированная поверхность (пол), над которой расположены три платоновых тела. Сверху находятся несколько источников света. На каждом ребре многогранника располагается определенное количество точечных источников света. Грани тел обладают зеркальным и прозрачным эффектом. За счет многократного переотражения лучей внутри тела, возникает эффект бесконечности. Камера. Камера выполняет облет сцены согласно определенным законам. В цилиндрических координатах (r , φ , z), положение и точка направления камеры в момент времени t определяется следующим образом:

$$r_c(t) = r_c^0 + A_c^r \sin(\omega_c^r t + p_c^r);$$

$$z_c(t) = z_c^0 + A_c^z \sin(\omega_c^z t + p_c^z);$$

$$\varphi_c(t) = \varphi_c^0 + \omega_c^\varphi t;$$

$$r_n(t) = r_n^0 + A_n^r \sin(\omega_n^r t + p_n^r);$$

$$z_n(t) = z_n^0 + A_n^z \sin(\omega_n^z t + p_n^z);$$

$$\varphi_n(t) = \varphi_n^0 + \omega_n^\varphi t;$$

$$\text{где } t \in [0, 2\pi]$$

Требуется реализовать алгоритм обратной трассировки лучей (<http://www.ray-tracing.ru/>) с использованием технологии CUDA. Выполнить покадровый рендеринг сцены. Для устранения эффекта «зубчатости», выполнить сглаживание (например с помощью алгоритма SSAA). Полученный набор кадров склеить в анимацию любым доступным программным обеспечением. Подобрать параметры сцены, камеры и освещения таким образом, чтобы получить наиболее красочный результат. Провести сравнение производительности гри и сри (т.е. дополнительно нужно реализовать алгоритм без использования CUDA).

Варианты заданий: на сцене должны располагаться три тела:

1. Тетраэдр, Гексаэдр, Октаэдр
2. Тетраэдр, Гексаэдр, Додекаэдр
3. Тетраэдр, Гексаэдр, Икосаэдр
4. Тетраэдр, Октаэдр, Додекаэдр
5. Тетраэдр, Октаэдр, Икосаэдр
6. Тетраэдр, Додекаэдр, Икосаэдр
7. Гексаэдр, Октаэдр, Додекаэдр
8. Гексаэдр, Октаэдр, Икосаэдр
9. Гексаэдр, Додекаэдр, Икосаэдр
10. Октаэдр, Додекаэдр, Икосаэдр

Программа должна принимать на вход следующие параметры:

1. Количество кадров.
2. Путь к выходным изображениям. В строке содержится спецификатор %d, на место которого должен подставляться номер кадра. Формат изображений

- соответствует формату, описанному в лабораторной работе 2.
3. Разрешение кадра и угол обзора в градусах по горизонтали.
 4. Параметры движения камеры: g_1^0 , z_1^0 , φ_1^0 , A_1^r , A_1^z , ω_1^r , ω_1^z , ω_1^0 , p_1^r , p_1^z и g_2^0 , z_2^0 , φ_2^0 , A_2^r , A_2^z , ω_2^r , ω_2^z , ω_2^0 , p_2^r , p_2^z .
 5. Параметры тел: центр тела, цвет (нормированный), радиус (подразумевается радиус сферы, в которую можно было бы вписать тело), коэффициент отражения, коэффициент прозрачности, количество точечных источников света на ребре.
 6. Параметры пола: четыре точки, путь к текстуре, оттенок цвета и коэффициент отражения.
 7. Количество (не более четырех) и параметры источников света: положение и цвет.
 8. Максимальная глубина рекурсии и квадратный корень из количества лучей на один пиксель (для SSAA).

Пример входных данных:

```
1024
/home/checker/pgp/ivanov/kp/out/img_%d.data
640 480 120
7.0 3.0 0.0 2.0 1.0 2.0 6.0 1.0 0.0 0.0
2.0 0.0 0.0 0.5 0.1 1.0 4.0 1.0 0.0 0.0
2.0 0.0 0.0 1.0 0.0 0.0 1.0 0.9 0.1 10
0.0 2.0 0.0 0.0 1.0 0.0 0.75 0.8 0.2 5
0.0 0.0 0.0 0.0 0.7 0.7 0.5 0.7 0.3 2
-5.0 -5.0 -1.0 -5.0 5.0 -1.0 5.0 5.0 -1.0 5.0 -5.0 -1.0 ~/floor.data 0.0 1.0 0.0 0.5
2
-10.0 0.0 10.0 1.0 1.0 1.0
1.0 0.0 10.0 0.0 0.0 1.0
10 16
```

Программа должна поддерживать следующие ключи запуска:

--сри Для расчетов используется только центральный процессор

--gpi Для расчетов задействуется видеокарта

--default В stdout выводится конфигурация входных данных (в формате описанном ранее), при которой получается наиболее красочный результат, после чего программа завершает свою работу.

Запуск программы без аргументов подразумевает запуск с ключом --gpi.

В процессе работы программа должна выводить в stdout статистику в формате: {номер кадра}\t{время на обработку кадра в миллисекундах}\t{общее количество лучей}\n

Программу необходимо отправить на чекер с темой письма pgr:kp

Программное и аппаратное обеспечение

OS

Семейство	Windows
Версия	10.0.19045
Релиз	10
Платформа	Windows-10-10.0.19045
Архитектура	AMD64

CPU

Модель	AMD Ryzen 5 5600H with Radeon Graphics
Архитектура	AMD64
Разрядность	64
Частота (паспортная)	3.2940 GHz
Частота (факт.)	3.3010 GHz
Ядер (физ.)	6
Потоков (лог.)	12

RAM

Всего	15.35 GB
Доступно	4.35 GB
Swap	6.00 GB

Диски

Диск	ФС	Всего, GB	Используй- вано, GB	Свободно, GB	Используй- вано, %
C:\	NTFS	470.94	391.13	79.82	83.05
D:\	FAT32	0.50	0.00	0.50	0.00

GPU

Compute capability	8.6
Name	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Total Global Memory	4294443008
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	65536
Multiprocessor's count	20

Используемая IDE: Visual Studio Code

Версия компилятора:

```
C:\Program Files\Microsoft Visual Studio\2022\Community>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_13:58:20_Pacific_Daylight_Time_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

Метод решения

Для визуализации трехмерной сцены в данной работе реализован метод **обратной трассировки лучей (Ray Tracing)**. Суть метода заключается в построении изображения путем отслеживания хода лучей от наблюдателя (камеры) через пиксели экрана к объектам сцены. Расчеты реализованы с поддержкой параллельных вычислений на графическом процессоре (GPU) с использованием технологии NVIDIA CUDA, а также с возможностью выполнения на центральном процессоре (CPU).

1. Представление сцены и геометрические примитивы

Сцена представляется в виде линейного массива треугольников $\text{Triangle} \{ a, b, c, \text{albedo} \}$, где a, b, c — вершины треугольника, а albedo — базовый цвет поверхности. Перед началом рендеринга на CPU формируется список треугольников triHost , включающий:

- **плоскость пола**, заданную четырьмя точками и разбитую на два треугольника;
- **тетраэдр** (4 треугольника), построенный из четырёх вершин вида $(\pm 1, \pm 1, \pm 1)/\sqrt{3}$ с последующим масштабированием радиусом и переносом в центр тела;
- **икосаэдр** (20 треугольников), вершины которого задаются через золотое сечение $\phi = (1 + \sqrt{5})/2$, нормализуются на единичную сферу и затем масштабируются;
- **додекаэдр**, который строится как двойственный к икосаэдру: центры граней икосаэдра формируют вершины додекаэдра, а каждая пятиугольная грань триангулируется «всером» (3 треугольника на пятиугольник).

Итоговая сцена хранится без ускоряющих структур, поэтому для каждого луча выполняется полный перебор всех треугольников.

2. Модель камеры и генерация первичных лучей

Используется модель **камеры-обскуры (Pinhole camera)**. Для каждого кадра задаются позиция камеры P_cP_c и точка наблюдения P_vP_v . На их основе строится ортонормированный базис камеры.

Если $bz^{\rightarrow}$ почти параллелен $UP^{\rightarrow}$, выбирается запасной вектор вверх $(0, 1, 0)$, чтобы избежать вырожденности.

Далее для каждого пикселя (x, y) вычисляются нормированные координаты экранной плоскости $v_x, v_y \in [-1; 1]$ с учётом соотношения сторон h/w . Компонента v_z определяется через угол обзора: $v_z = 1/\tan(\text{FOV} \cdot \pi/360)$

Направление первичного луча в мировых координатах вычисляется как:

$$D^{\rightarrow} = \text{norm}(bx^{\rightarrow} vx + by^{\rightarrow} vy + bz^{\rightarrow} vz)$$

Луч задаётся параметрически:

$$P(t)=O+tD^{\rightarrow}, O=P_c$$

Таким образом, каждому пикселю соответствует один первичный луч.

3. Поиск пересечений (Ray–Triangle Intersection)

Для нахождения пересечения луча с треугольником используется алгоритм **Мёллера—Трумбора**. Он позволяет быстро определить факт пересечения и вычислить расстояние t вдоль луча без предварительного построения уравнения плоскости треугольника.

Пусть треугольник задан вершинами V_0, V_1, V_2 , тогда луч пересекает треугольник при существовании барицентрических координат u, v, v , удовлетворяющих:

$$O+tD^{\rightarrow}=(1-u-v) V_0+uV_1+vV_2$$

Условия корректного попадания: $u \geq 0, v \geq 0, u+v \leq 1, t > 0$

В реализации дополнительно используется отсечение $t > 10^{-4}$ для подавления самопересечений и численных артефактов. Для каждого луча перебираются все треугольники сцены, и выбирается **ближайшее** пересечение (минимальное t).

Нормаль поверхности в точке пересечения вычисляется по векторному произведению рёбер: $N^{\rightarrow} = \text{norm}((V_1 - V_0) \times (V_2 - V_0))$ и разворачивается в сторону камеры (если $N^{\rightarrow} \cdot D^{\rightarrow} > 0$, то $N^{\rightarrow} = -N^{\rightarrow}$).

4. Модель освещения и расчёт теней

В работе используется локальная модель освещения с учётом **фоновой (ambient)** и **диффузной (Lambert)** составляющих от одного точечного источника света L_p . В точке пересечения p строится вектор на источник света: $L^{\rightarrow} = L_p - p, L = L^{\rightarrow} / |L^{\rightarrow}|$

Диффузная составляющая вычисляется по закону Ламберта: $\text{diff} = \max(0, N^{\rightarrow} \cdot L)$

и добавляется к фоновому освещению с постоянным коэффициентом: $\text{ambient} = 0.18$

Итоговый цвет пикселя рассчитывается по компонентам RGB как:

$$C = C_{\text{obj}} \cdot \text{ambient} + (C_{\text{obj}} \cdot \text{Clight}) \cdot \text{diff} \cdot S_{\text{shadow}}$$

где C_{obj} — цвет материала треугольника, Clight — цвет источника света, $S_{\text{shadow}} \in \{0, 1\}$ — коэффициент видимости света.

Расчёт теней

Для проверки затенения из точки пересечения выпускается **теневогой луч** к источнику света. Чтобы избежать самопересечения с той же поверхностью, начало луча смещается вдоль нормали:

$$p_{\text{shadow}} = p + \varepsilon N^{\rightarrow}, \varepsilon = 10^{-3}$$

Если теневогой луч пересекает любой треугольник на расстоянии, меньшем расстояния до источника света, точка считается в тени, и диффузная составляющая зануляется: $S_{\text{shadow}} = 0 \Rightarrow \text{diff} = 0$

Если пересечений нет, точка освещена: $S_{\text{shadow}} = 1$

5. Параллелизация вычислений (CUDA)

Алгоритм трассировки лучей является естественно параллельным, так как цвет каждого пикселя вычисляется независимо. На GPU используется CUDA-ядро `renderKernel`, где:

- изображение состоит из $w \cdot h$ пикселей;

- каждый поток обрабатывает несколько пикселей с шагом $\text{stride} = \text{blockDim} \cdot \text{grid}$ (циклическое распределение нагрузки);
- для каждого пикселя внутри потока формируется первичный луч и выполняется трассировка с учётом теней.

Данные сцены (массив треугольников) выделяются один раз до рендеринга и доступны на устройстве. После завершения ядра буфер изображения копируется на CPU и сохраняется в файл. Также в программе ведётся счётчик количества выпущенных лучей (первичные + теньевые), что позволяет оценивать нагрузку при измерениях производительности.

6. Анимация (движение камеры)

Для построения последовательности кадров положение камеры и точки наблюдения задаются параметрически через тригонометрические функции. Для кадра `frame` вычисляется параметр времени: $t = 2\pi \cdot \text{frame} / \text{framesAmount}$

Далее радиус, угол и высота камеры изменяются по законам вида: $r(t) = r_0 + \arcsin(\omega r t + p_r)$, $\phi(t) = \phi_0 + \omega_\phi t$, $z(t) = z_0 + a_z \sin(\omega_z t + p_z)$

Это обеспечивает плавное циклическое движение камеры (орбитальный облет и колебания по высоте), благодаря чему получается анимация сцены без изменения самой геометрии.

Описание программы

Программа реализована в виде **единого файла компиляции** `run.cu`, содержащего как код для центрального процессора (**Host**), так и код для графического ускорителя (**Device**). Монолитная структура выбрана для упрощения сборки и для возможности использовать одни и те же вспомогательные функции (векторная алгебра, генерация лучей, пересечения) в вариантах CPU/GPU без дублирования кода. Для этого часть функций помечена атрибутами `__host__ __device__`, что позволяет компилировать их и для x86/x64, и для CUDA-устройства.

Структуры данных и форматы представления

Для хранения данных используются простые структуры, корректно работающие как на CPU, так и на GPU:

- **struct Vec3** — базовый тип для представления трёхмерных векторов и точек (x,y,z). Использует тип `float`, содержит конструкторы, а также поддерживается набор операций (сложение/вычитание/умножение на скаляр, `dot`, `cross`, `normalize`).
- **struct Pixel** — цвет пикселя в формате RGBA (каналы `uint8_t r,g,b,a`). Для перехода между вычислениями в `float` и 8-битным форматом реализованы функции `rgbToPixel()` и `pixelToRGB()`, а также ограничение диапазона через `clamp01()`.
- **struct Triangle** — геометрический примитив сцены: три вершины a,b,c (типа `Vec3`) и цвет поверхности `albedo` (типа `Pixel`). Сцена хранится как линейный массив таких треугольников.

- **struct FramesParams** — параметры последовательности кадров: количество кадров `amount`, шаблон имени выходных файлов `outPattern`, разрешение `w×h` и угол обзора `fovDeg`.
- **struct CameraParams** — параметры движения камеры и точки наблюдения (радиусы, углы, высоты, амплитуды/частоты/фазы), используемые в функции `evalCamera()` для анимации.
- **struct BodyParams** — параметры тел сцены: центр `center`, базовый цвет `color`, радиус `radius` и дополнительные поля (например, отражение/прозрачность/параметры рёбер), которые в текущем упрощённом варианте не используются в расчёте освещения, но сохранены в формате входных данных.
- **struct FloorParams** — параметры пола: четыре вершины прямоугольника `p1..p4`, строковое поле `texPath` и цвет `color`. В данном варианте текстуры не применяются, пол окрашивается константным цветом.
- **struct Light** — параметры точечного источника света: позиция `pos` и цвет `color`.
- **struct TailParams** — дополнительные параметры (например, `maxDepth`, `sqrt_rpp`), которые в текущей версии читаются из входного потока для совместимости формата, но непосредственно в рендеринге не участвуют.

Чтобы упростить ввод параметров, для основных структур перегружен оператор `operator>>`, и конфигурация сцены читается из **stdin** в фиксированном формате.

Вспомогательные функции

В программе реализован набор функций, обеспечивающих базовую математику и сервисные операции:

- операции над `Vec3` и функции линейной алгебры: `dot`, `cross`, `length`, `normalize`;
- построение базиса камеры `buildCameraBasis()` и генерация направления первичного луча для пикселя `generateRayDir()` (используются и в CPU-версии, и в GPU-версии);
- преобразования цвета: `rgbToPixel()`, `pixelToRGB()`, чтение цвета в нормированном виде `readNormColor()`;
- формирование имени выходного файла `formatFramePath()` (поддержка шаблона с `%d` либо добавление суффикса `_frame`);
- запись результата в бинарный файл `writeImage()`. Формат кадра: **int32 w, int32 h, затем массив Pixel[w*h]**.

Для обработки CUDA-ошибок используется макрос **CSC(call)**, который проверяет код возврата CUDA-функций и при ошибке выводит сообщение в `stderr`, после чего завершает программу.

Построение сцены

Сцена собирается на CPU в контейнер `std::vector<Triangle>` и затем копируется в память, доступную GPU. Для добавления объектов используются функции:

- `addFloor()` — формирует пол из двух треугольников по четырём вершинам прямоугольника;
- `addTetrahedron()` — строит тетраэдр из 4 треугольников;

- `addIcosahedron()` — строит икосаэдр из 20 треугольников (вершины и грани задаются в `buildIcosaModel()`);
- `addDodecahedron()` — строит додекаэдр как двойственный к икосаэдру: центры граней икосаэдра формируют вершины додекаэдра, а каждая пятиугольная грань триангулируется «веером» (в итоге 36 треугольников).

Треугольники сцены выделяются через `cudaMallocManaged` (Unified Memory), после чего элементы из `std::vector` копируются в этот массив и используются как на CPU, так и на GPU.

Алгоритмическое ядро (основная логика трассировки)

Ключевые функции трассировки реализованы так, чтобы работать в обоих режимах:

1. `intersectTriangle()` — пересечение луча с одним треугольником по алгоритму Мёллера—Трумбора. Возвращает факт попадания, расстояние `tHit` и нормаль `nHit`. Нормаль разворачивается в сторону камеры.
2. `anyHitShadow()` — проверка тени: из точки пересечения выпускается луч к источнику света, и выполняется перебор всех треугольников. Если найдено пересечение ближе источника, точка считается затенённой.
3. `traceRay()` — трассировка одного луча: выполняет поиск ближайшего пересечения (полный перебор `triCount`), вычисляет освещённость (`ambient` + диффузная составляющая по Ламберту) и учитывает тень через `anyHitShadow()`. Результатом является итоговый цвет `Pixel`.

Также присутствует CPU-реализация рендера:

- `renderCPU()` — двойной цикл по пикселям, генерация луча на каждый пиксель, вызов `traceRay()`, запись результата в буфер `std::vector<Pixel>`.

CUDA-ядро и схема параллелизации

Основной вычислительный код на GPU реализован в ядре:

```
__global__ void renderKernel(Pixel* out, int32_t w, int32_t h, float fovDeg, Vec3 camPos, Vec3 camView,
Light light, const Triangle* tris, int triCount, uint64_t* rayCounter)
```

Принцип работы ядра:

1. Используется **одномерная** схема распараллеливания. Индекс потока: `idx=blockIdx.x*blockDim.x+threadIdx.x`

общий шаг по пикселям: `offset=blockDim.x*gridDim.x`
2. Каждый поток обрабатывает несколько пикселей в цикле `for (i = idx; i < w*h; i += offset)`.
3. Для кадра в каждом потоке вычисляется базис камеры `buildCameraBasis()`, затем для каждого пикселя строится направление луча `generateRayDir()`, и цвет рассчитывается вызовом `traceRay()`.

4. Результат записывается в линейный буфер `out[i]` в формате `Pixel`.

Общий поток выполнения (функция `main`)

Программа поддерживает режимы запуска через аргументы командной строки:

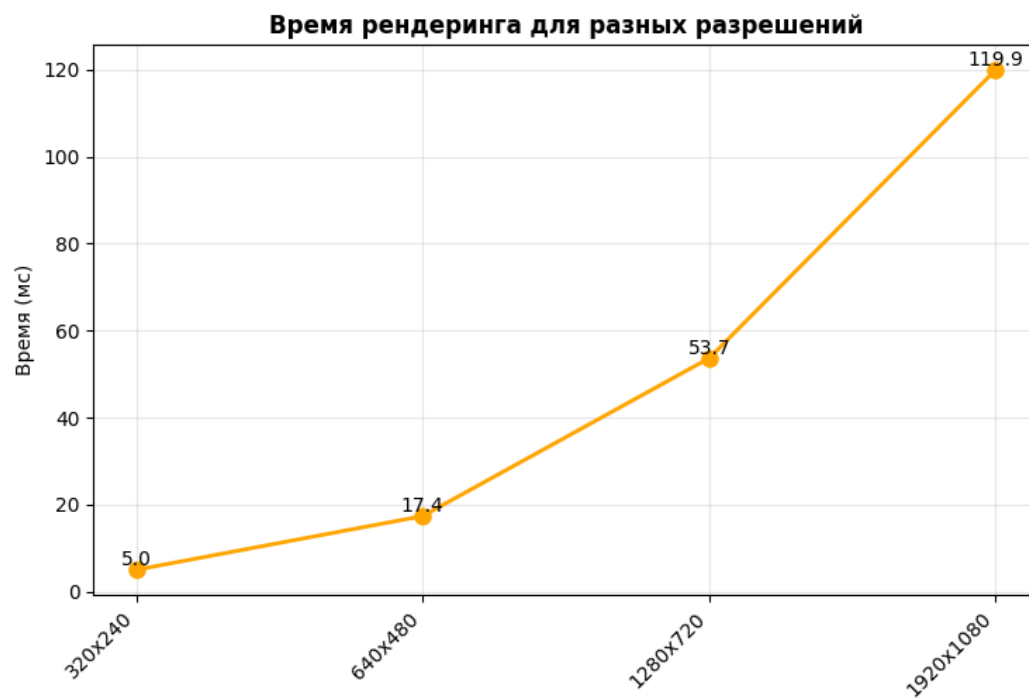
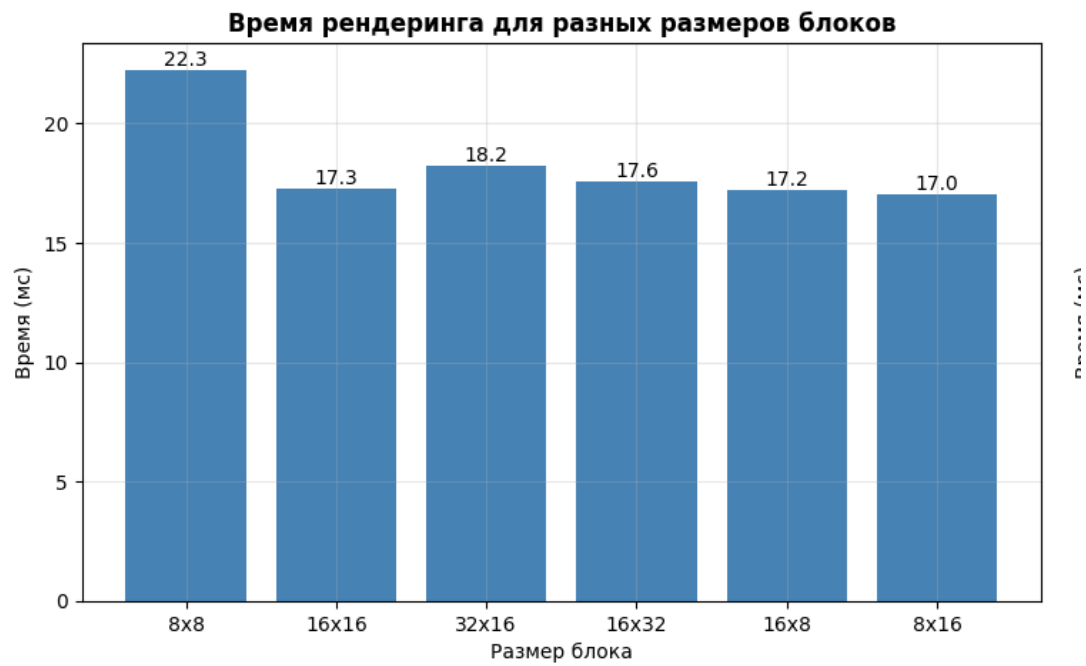
- `--gpu` (по умолчанию используется GPU),
- `--cpu` (принудительный запуск CPU-версии),
- `--default` (выводит на экран пример конфигурации входных данных в корректном формате и завершает работу).

Основной сценарий работы:

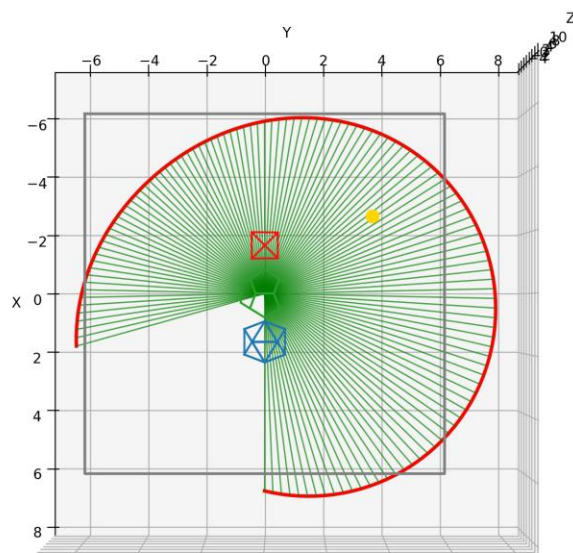
1. Чтение параметров из `stdin` в структуры `FramesParams`, `CameraParams`, `BodyParams` (3 тела), `FloorParams`, массив источников света `Light`, а также `TailParams`.
2. Проверка корректности размеров изображения и количества кадров.
3. Сборка сцены в `std::vector<Triangle>` и копирование в массив `Triangle*`, выделенный через `cudaMallocManaged`.
4. Выделение памяти под выходной буфер: на GPU — `cudaMalloc(d_out)` и счётчик лучей `cudaMallocManaged(d_rays)`, на CPU — `std::vector<Pixel> hostPixels`.
5. Цикл по кадрам:
 - вычисление положения камеры и точки наблюдения `evalCamera()`;
 - запуск рендера (GPU: `renderKernel<<<256,256>>> + cudaDeviceSynchronize + cudaMemcpy` на `host`; либо CPU: `renderCPU()`);
 - запись кадра в файл через `writeImage()` (имя формируется `formatFramePath()` по шаблону);
 - вывод в консоль строки вида: `frame \t time_ms \t rays`.
6. Освобождение ресурсов (`cudaFree` для выделенных CUDA-буферов и сцены).

Такое построение программы обеспечивает единый код трассировки для CPU/GPU, простую структуру сцены (массив треугольников) и воспроизводимый формат ввода/вывода, удобный для пакетного рендеринга анимации в последовательность кадров.

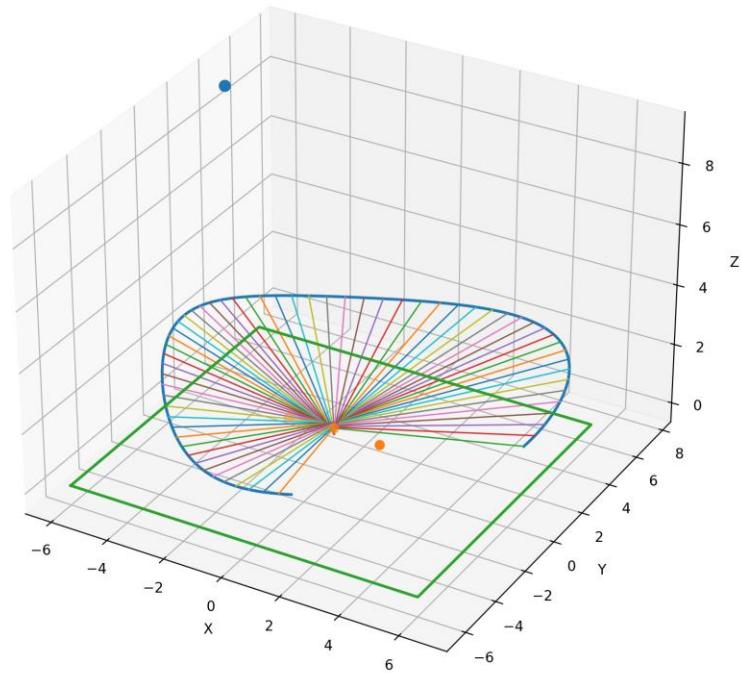
Исследовательская часть и результаты



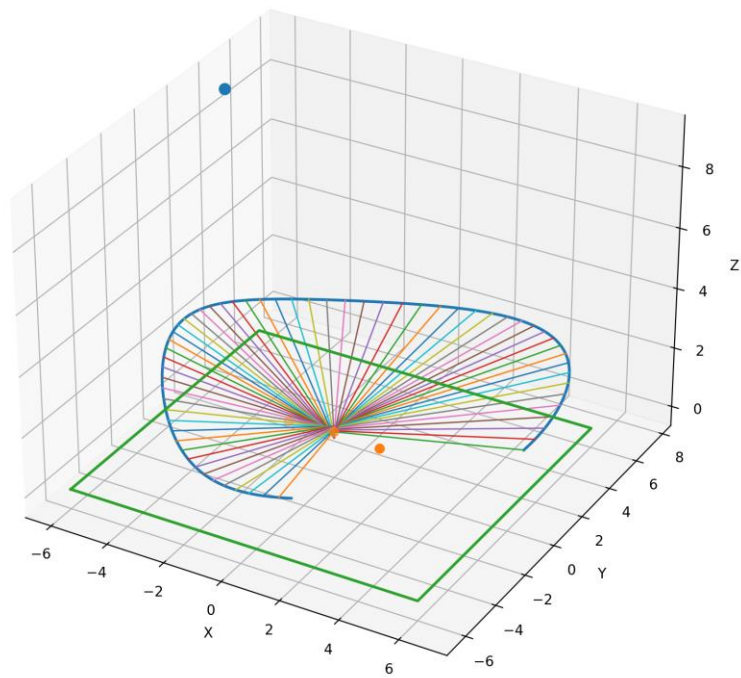
Camera/View paths + wireframe bodies

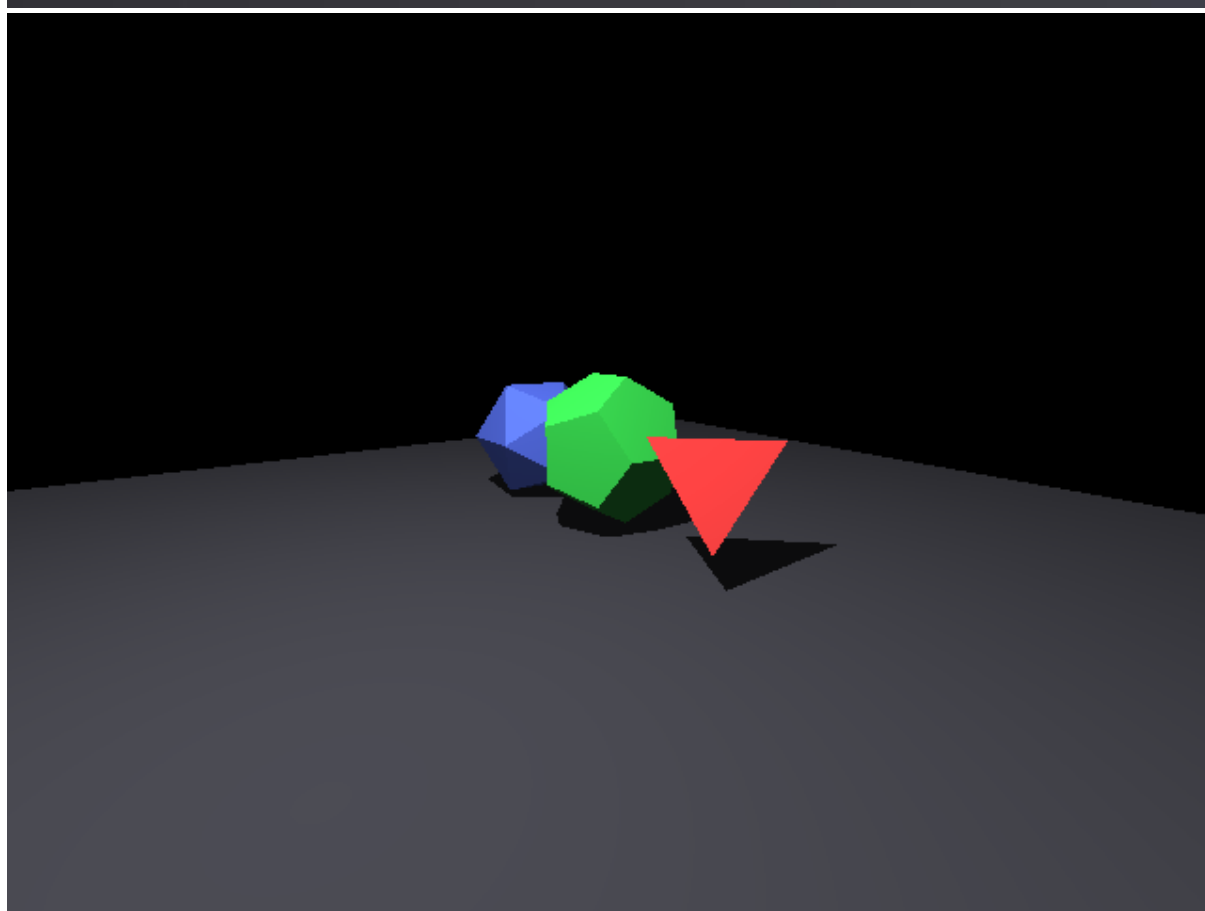
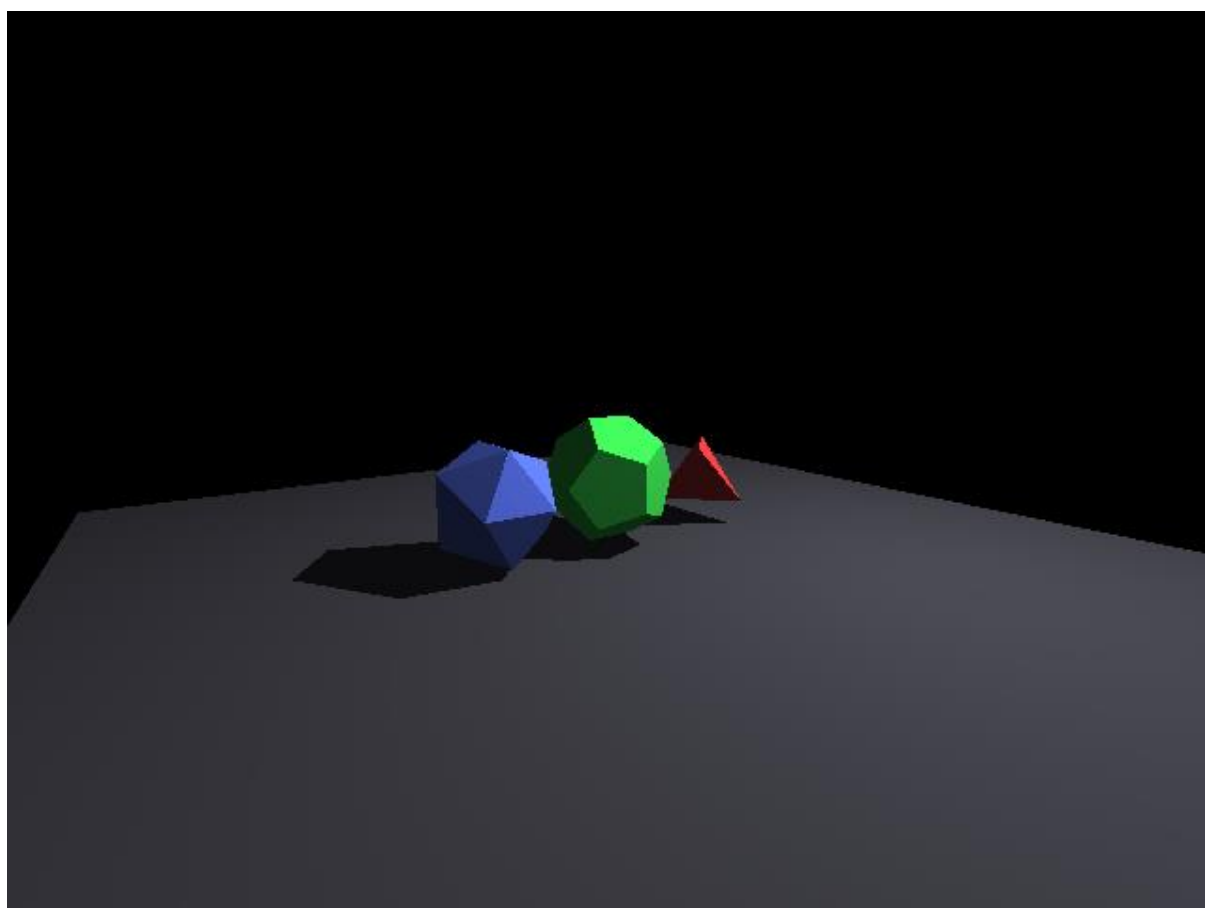


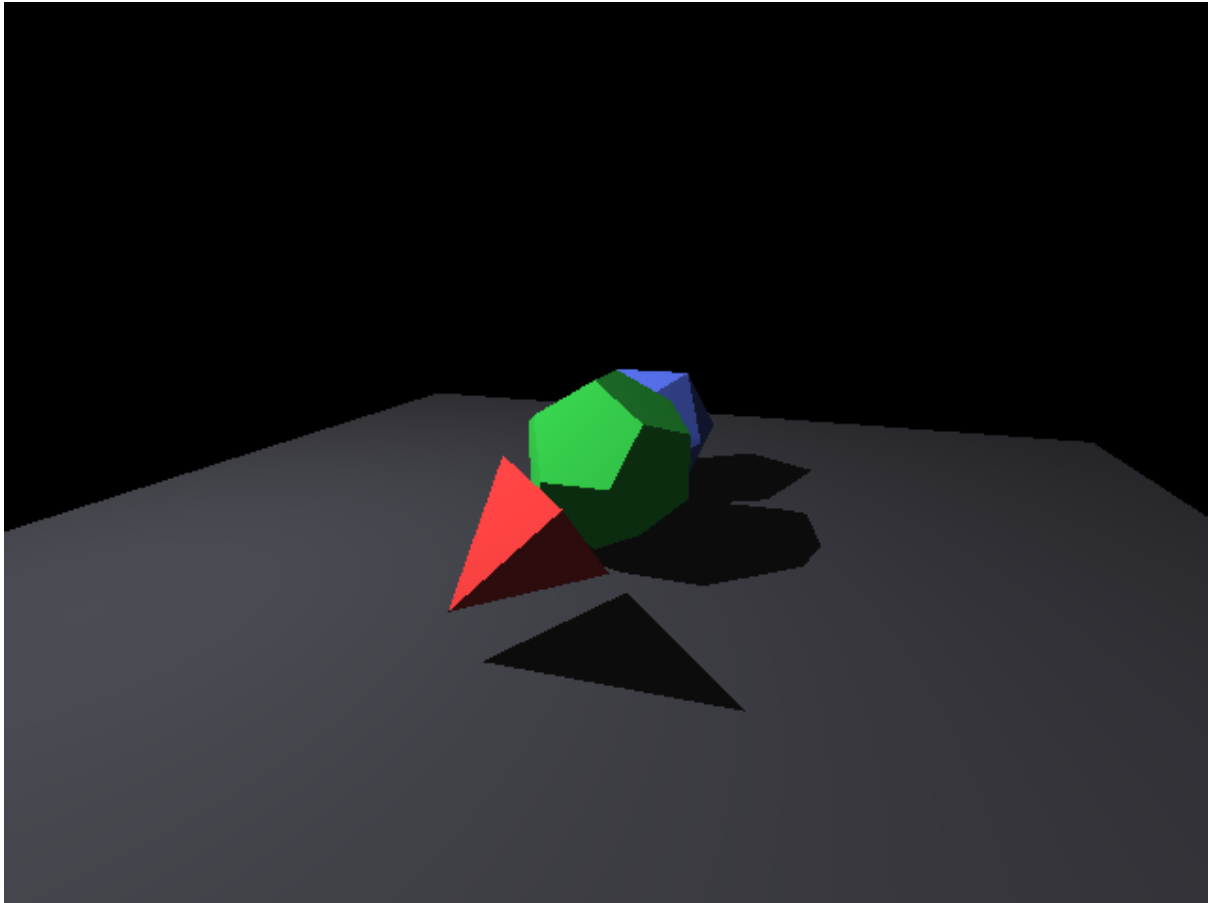
Camera / View paths + Light



Camera / View paths + Light







Входные данные, на которых наиболее красочный результат:

```
120
./out/img_%d.data
640 480 90
6.5 2.2 0.0 1.2 0.6 1.0 2.0 0.8 0.0 0.0
0.0 0.9 0.0 0.0 0.2 1.2 1.5 0.8 0.0 0.0
-1.6 0.0 0.8 0.95 0.25 0.25 0.75 0.0 0.0 0
0.0 0.0 0.9 0.25 0.95 0.35 0.85 0.0 0.0 0
1.6 0.0 0.8 0.35 0.45 0.95 0.80 0.0 0.0 0
-6.0 -6.0 0.0 -6.0 6.0 0.0 6.0 6.0 0.0 6.0 -6.0 0.0 none 0.25 0.25 0.28 0.0
1
-2.5 3.5 5.5 1.0 1.0 1.0
0 2
```

Сравнение gpu/cpu:

Номер кадра	Количество лучей = 1356000	
	Время работы cpu, ms	Время работы gpu, ms
1	23.49	1296.61
2	25.11	1286.27
3	26.91	1302.05
4	24.22	1304.28
5	20.61	1283.95
6	24.42	1269.52
7	22.11	1273.27
8	19.74	1243.82
9	21.86	1255.12
10	20.65	1241.43
11	22.02	1237.25
12	20.73	1232.83
13	18.93	1223.80
14	17.81	1231.12
15	19.09	1224.60

Выводы

В работе реализован простой рендерер трёхмерной сцены на основе обратной трассировки лучей, рассчитанный на параллельное выполнение на GPU (CUDA) и имеющий CPU-вариант для сравнения и проверки корректности. Сцена хранится как линейный массив треугольников, пересечения вычисляются по алгоритму Мёллера—Трумбора, а освещение задаётся локальной моделью Ambient + диффузная составляющая по Ламберту с учётом теней (выпуск теневого луча к точечному источнику света). Визуализация выполняется без рекурсии и сложных эффектов (отражений/текстур), что соответствует упрощённому варианту задания.

Параллелизация на GPU реализована в ядре renderKernel: потоки независимо вычисляют цвет пикселей, распределяя работу по всему изображению с шагом, равным общему числу потоков. Программа поддерживает генерацию последовательности кадров — положение камеры и точки наблюдения вычисляется параметрически функцией evalCamera(), что обеспечивает плавное движение камеры вокруг объектов. Каждый кадр сохраняется в бинарный файл (ширина, высота и массив пикселей RGBA), а в консоль выводятся замеры времени и счётчик обработанных лучей, что позволяет оценивать производительность в лучах/сек.

Сравнение CPU и GPU версий демонстрирует существенное ускорение на видеокарте за счёт естественной независимости вычислений по пикселям. Основное ограничение текущей реализации — полный перебор всех треугольников для каждого луча, из-за чего

рост сложности сцены напрямую увеличивает время рендеринга. Потенциальные направления улучшений включают добавление ускоряющих структур (BVH/сетки), более точные замеры на стороне GPU (CUDA events) и расширение модели освещения при необходимости.

Литература

- NVIDIA Corporation. CUDA C Programming Guide. Version 12.0. NVIDIA, 2023. – URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>;
- NVIDIA Corporation. CUDA C Best Practices Guide. NVIDIA, 2023. – URL: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>;
- <https://www.deepseek.com/>
- Лекции Морозова А.Ю.